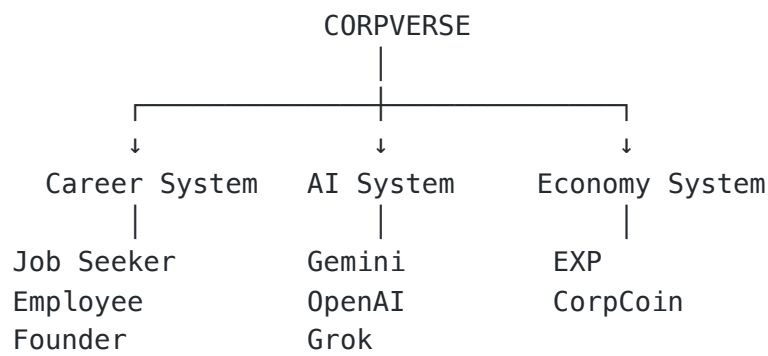


CorpVerse v1.0 — Complete Rules & Technical Specification

1. Core philosophy

CorpVerse has three different systems working together:



Career System

Controls:



AI System

Controls:

- Resume Analysis
- ATS
- Interview
- Task Generation
- Task Evaluation
- Feedback
- Company Simulation

Economy System

Controls:

EXP
CorpCoin
Level
Promotion
Company Profit/Loss

These three systems should be **separate backend modules**.

2. User roles

There are actually two categories of roles.

A. Career roles

These are progression roles.

JOB_SEEKER
EMPLOYEE
FOUNDER

A user normally starts as:

JOB_SEEKER

and can progress:

JOB_SEEKER
↓
EMPLOYEE
↓
FOUNDER

If an employee is fired:

EMPLOYEE
↓
JOB_SEEKER

If a founder's company collapses:

FOUNDER



JOB_SEEKER

B. Platform roles

These are privileged system roles.

ADMIN

AI_MANAGER

They should **not be part of the normal career progression**.

For example:

User A

careerRole = EMPLOYEE

platformRole = NONE

User B

careerRole = FOUNDER

platformRole = NONE

Admin

careerRole = NONE

platformRole = ADMIN

AI Manager

careerRole = NONE

platformRole = AI_MANAGER

This is much cleaner than putting everything into one `role` field.

3. Supported career domains

For CorpVerse v1:

SOFTWARE_ENGINEERING

CLOUD_ENGINEERING

AI_ENGINEERING

Do **not** add more domains automatically.

The database should still be designed so Admin can add domains later.

4. Job levels

Every employee has a level.

I recommend:

Level	Name	Required Total EXP
1	Intern / Beginner	0
2	Junior	500
3	Junior+	1,200
4	Associate	2,000
5	Mid-Level	3,000
6	Mid-Level+	4,500
7	Senior	6,500
8	Senior+	9,000
9	Lead	12,000
10	Principal	16,000

These are **total accumulated EXP**, not EXP earned in one job.

5. Founder Mode unlock

This is one of the limits you originally left undecided.

I recommend:

Founder unlock requirement

12,000 total EXP

That corresponds to approximately:

Why?

Founder mode should feel like:

"I have experienced enough of the simulated corporate world to now try running one."

It should not be available to a new user after a few tasks.

Therefore:

```
TOTAL_EXP >= 12,000
      ↓
Founder Mode Available
```

But reaching 12,000 does **not automatically turn the user into Founder**.

Instead:

```
Employee
  ↓
12,000 EXP
  ↓
"Unlock Founder Mode"
  ↓
User confirms
  ↓
Founder
```

6. Founder starting CorpCoin

Set:

1,000 CorpCoin

when Founder Mode is unlocked.

The Founder receives the 1,000 CorpCoin **only once**.

They cannot repeatedly:

unlock → receive 1,000
leave → unlock → receive 1,000

The database should maintain:

```
founderModeUnlockedAt  
founderStarterCoinGranted = true
```

7. CorpCoin rules

CorpCoin is the company's economic currency.

It should NOT be mixed with EXP.

EXP

Represents:

| Career experience.

CorpCoin

Represents:

| Founder/company capital.

Starting Founder company

Founder gets:

1,000 CorpCoin

Initial company creation costs:

100 CorpCoin

Remaining:

This prevents free unlimited company creation.

8. AI bot costs

For the first version, keep the economy simple.

Bot categories

HIRING_BOT
TASK_BOT
EVALUATION_BOT

Initial prices

Bot	Cost
Hiring AI	250 CorpCoin
Task AI	250 CorpCoin
Evaluation AI	250 CorpCoin
Advanced Hiring AI	400 CorpCoin
Advanced Task AI	400 CorpCoin
Advanced Evaluation AI	400 CorpCoin

For MVP, however, **every company needs the three basic bots.**

So:

Company creation = 100
Hiring Bot = 250
Task Bot = 250
Evaluation Bot = 250

Total = 850 CorpCoin

Founder starts with 1,000.

Remaining:

That gives the founder a small buffer.

9. Founder company limit

A Founder can own:

1 active company

in v1.

Do not allow:

Founder → Company A

Founder → Company B

Founder → Company C

This makes the economy much easier to balance.

Future versions can introduce multiple companies.

10. Company employee limit

Initial company:

Maximum 20 employees

This prevents one founder/company from immediately dominating the entire platform.

Later, company upgrades can increase this.

For v1:

`MAX_EMPLOYEES = 20`

11. Employee warning system

This was one of your undecided values.

I recommend:

4 warnings → termination/demotion decision

But don't make every bad task automatically produce a warning.

Use:



Score

- 0–39 = Poor
- 40–59 = Needs Improvement
- 60–74 = Acceptable
- 75–89 = Good
- 90–100 = Excellent

Warning rules

Score	Result
90–100	Excellent
75–89	Good
60–74	Normal
40–59	Performance issue
0–39	Warning candidate

A warning should require **serious/repeated poor performance**, not a single slightly-below-average answer.

12. Exact termination rule

Use:

```
Warning 1
Warning 2
Warning 3
Warning 4
  ↓
Performance Review
  ↓
Termination OR Demotion
```

However, I recommend that the **4th warning does not automatically fire everyone.**

Instead:

```
warningCount >= 4
  ↓
Employment Review
  ↓
AI + company rules
  ↓
DEMOTION or TERMINATION
```

This gives you room to implement realistic corporate decisions.

13. Warning decay

Otherwise a user could receive four warnings over six months and eventually be fired for old mistakes.

So:

Warning expiration = 30 days

Example:

```
Warning 1
Day 0

Warning 2
Day 5
```

Warning 3
Day 10

Warning 1 expires
Day 30

Only active warnings count toward termination.

14. Daily employee tasks

Each active employee receives:

1 primary task per day

Optional:

1 bonus task per day

So maximum:

2 tasks/day

Don't initially generate 5–10 AI tasks every day.

It increases:

- AI cost
- database load
- complexity
- user fatigue

15. Task EXP

Task EXP range:

10–100 EXP

based on:

Difficulty
Task **quality**
Completeness
Reasoning
Technical **correctness**

Suggested difficulty multiplier:

Difficulty	Maximum EXP
Easy	30
Medium	60
Hard	100

The AI **must not be allowed to return arbitrary EXP** such as 3,847.

The backend should clamp it:

```
0 <= awardedExp <= task.maxExp
```

The AI recommends the score.

Backend validates the score.

This is extremely important.

16. EXP should never be controlled directly by AI

Bad architecture:

AI says:
"Give user 5000 EXP"

Backend:
Okay.

Correct architecture:

AI **evaluates task**
↓

AI returns score 0–100

↓

Backend calculates EXP

↓

Backend updates EXP

For example:

```
task.maxExp = 60  
score = 82
```

```
earnedExp = calculatedByBackend(score, 60)
```

This prevents AI hallucination from breaking the economy.

17. Promotion

Promotion should not happen solely because AI says:

"Promote this user."

Backend determines eligibility.

Example:

```
EXP requirement  
+  
minimum performance  
+  
active warning count  
+  
minimum completed tasks
```

Example:

Level 4 → Level 5

```
Requirements:  
3,000 total EXP  
10 completed tasks  
Average score >= 70  
Active warnings <= 1
```

AI can provide a recommendation, but:

Backend owns the final decision.

18. Demotion

Demotion occurs after serious repeated poor performance.

Example:

4 active warnings

↓

Review

↓

Demotion

Demotion should reduce:

Job level

Position

Salary/compensation simulation

Responsibilities

But:

Never remove previously earned EXP.

For example:

User:

6,800 EXP

Level 7

Demoted to Level 6

EXP remains:

6,800

Otherwise the user could lose months of progress.

19. Firing

If terminated:

Employee
↓
JOB_SEEKER

The user keeps:

EXP
skills
resume
profile
career history
previous employment history

They lose:

current company position
active employee status
current job

This creates a realistic career history instead of resetting the account.

20. Founder failure

You mentioned:

If founder makes loss till one level then founder becomes job seeker.

Let's make this precise.

A company has:

Bankruptcy threshold = -1,000 CorpCoin equivalent

But because profit/loss should be separate from the wallet, use company financial health.

Recommended:

Company financial health <= -1,000
↓
Company Bankruptcy
↓

Founder loses company

↓

Founder → Job Seeker

However, the Founder should **not lose their personal EXP.**

They keep their career history.

21. Founder daily decisions

Founder receives:

1 business scenario per day

Example:

Employee turnover has increased by 15%. What action should the company take?

Founder chooses an option.

The simulation engine calculates:

Revenue

Expenses

Profit

Employee satisfaction

Productivity

Retention

Reputation

Do **not** allow an LLM to directly modify the database.

Correct:

Founder **Decision**

↓

Simulation **Engine**

↓

Deterministic **calculation**

↓

Database **update**

AI can generate the scenario and explain it.

The actual economic calculation should be controlled by backend code.

22. Company ranking

Ranking categories:

User

Highest EXP
Highest Level
Highest CorpCoin
Best Employee Performance
Best Founder

Company

Highest Profit
Highest Revenue
Largest Workforce
Best Employee Retention
Best Company Rating
Fastest Growing

Important

Ranking should be calculated from **stored backend values**, not AI-generated claims.

23. AI provider architecture — VERY IMPORTANT

You specifically pointed out:

Gemini, OpenAI and Grok APIs are different.

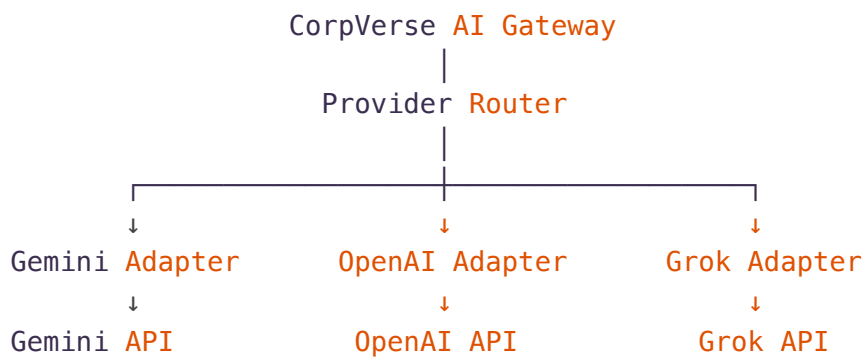
Correct.

Do not build this:

```
sendRequest(provider, prompt)
```

and assume every provider accepts the same request.

Instead build:



24. Why adapters are necessary

Each provider can differ in:

- authentication
- SDK
- endpoint
- model names
- request structure
- response structure
- streaming
- tool/function calling
- token limits
- rate limits
- error codes
- timeout behavior
- retry behavior
- structured output support

Therefore CorpVerse should have an **internal standard format**.

25. CorpVerse internal AI request

Your application should create something like:

AIRequest

```
{
  taskType,
  systemInstruction,
  userInput,
  context,
  outputSchema,
  temperature,
  maxTokens
}
```

But this is **CorpVerse's internal format**.

It is not sent directly to every provider.

26. Gemini adapter

Internally:

```
CorpVerse AIRequest
      ↓
GeminiAdapter
      ↓
Gemini-specific request
      ↓
Gemini API
      ↓
Gemini-specific response
      ↓
Normalize
```

27. OpenAI adapter

```
CorpVerse AIRequest
      ↓
OpenAIAdapter
      ↓
OpenAI-specific request
      ↓
OpenAI API
      ↓
OpenAI-specific response
```

↓
Normalize

28. Grok adapter

Same concept:

```
CorpVerse AIRequest
      ↓
GrokAdapter
      ↓
Grok-specific request
      ↓
Grok API
      ↓
Grok-specific response
      ↓
Normalize
```

29. Normalized CorpVerse AI response

Regardless of provider, CorpVerse should receive:

```
{
  success: true,
  provider: "gemini",
  model: "provider-model",
  requestId: "...",
  content: "...",
  structuredData: {},
  usage: {
    inputTokens: 0,
    outputTokens: 0,
    totalTokens: 0
  },
  latencyMs: 0
}
```

If OpenAI responds differently, the adapter converts it.

If Grok responds differently, the adapter converts it.

The rest of your application doesn't care.

30. Never hard-code provider logic into business logic

Bad:

```
HiringService
  ↓
Gemini API
```

Bad:

```
InterviewService
  ↓
OpenAI API
```

Bad:

```
TaskService
  ↓
Grok API
```

Correct:

```
HiringService
  ↓
AI Gateway
  ↓
Provider Router
  ↓
Selected Provider Adapter
```

This is one of the most important architectural decisions for CorpVerse.

31. AI Manager

AI Manager controls:

Provider
Model
Priority
Status
Rate limits
Daily limits
Failure count
Health
Fallback priority

Example:

Provider	Priority	Status
OpenAI	1	HEALTHY
Gemini	2	HEALTHY
Grok	3	HEALTHY

If OpenAI fails:

OpenAI
↓
Gemini
↓
Grok

But this should happen **through adapters**.

32. AI health states

Each provider can have:

HEALTHY
DEGRADED
RATE_LIMITED
TEMPORARILY_FAILED
DISABLED

Admin/AI Manager can see:

Provider
Current model
Health

Last successful request
Last failure
Failure count
Rate-limit status
Average latency

33. Fallback rules

Don't switch providers for every tiny error.

Recommended:

Timeout
Rate limit
Temporary provider error
Service unavailable
Network failure

→ fallback.

But:

Invalid user input
Invalid internal request
Authentication/configuration error

should generally be treated as configuration/application errors, not blindly retried across providers.

34. AI queue

Your idea:

If AI service fails, everything waits in queue.

Correct, but refine it.

AI Job
↓
Provider 1
↓
Failed?
↓

Retry according to policy
↓
Fallback Provider
↓
Still unavailable?
↓
QUEUE

Every AI job should have:

PENDING
PROCESSING
COMPLETED
FAILED
RETRYING
WAITING_FOR_PROVIDER
CANCELLED

35. AI retry limit

Set:

3 attempts per provider

Then move to fallback.

Example:

OpenAI
Attempt 1 ❌
Attempt 2 ❌
Attempt 3 ❌
↓
Gemini
Attempt 1 ✅

If all providers fail:

WAITING_FOR_PROVIDER

The job remains safe.

36. Important: AI should not directly control critical state

AI should **recommend/evaluate**.

Backend should decide:

EXP
CorpCoin
Level
Promotion
Demotion
Firing
Company balance
Bankruptcy
Permissions
Role

For example:

AI:
score = 82

Backend:
calculate EXP

Backend:
update user.exp

Backend:
check promotion eligibility

Not:

AI:
"Promote user and give 3000 EXP"

Backend:
Okay.

This single principle will prevent many future problems.

37. Resume storage

You said:

resume stored in MongoDB.

Use:

MongoDB GridFS

Architecture:

```
graph TD
    React --> Express
    Express --> ResumeService[Resume Service]
    ResumeService --> GridFS
    GridFS --> MongoDB
```

And extracted information:

```
ResumeFile
ResumeAnalysis
UserProfile
```

should be separate logical records.

Don't put the entire binary resume into the normal `users` document.

38. Resume processing

Flow:

```
graph TD
    Upload --> ValidateFile[Validate file]
    ValidateFile --> StoreGridFS[Store GridFS]
    StoreGridFS --> CreateProcessingJob[Create processing job]
```

↓
Extract text
↓
AI analysis
↓
Structured resume data
↓
Store analysis
↓
Show user review

Allowed resume formats for v1:

PDF
DOCX

Set maximum:

10 MB

39. Profile requirements

Mandatory:

Email
Password
Display Name
Domain
Skills
Resume

Optional:

GitHub
LinkedIn
Portfolio
Projects
Certifications

Do not make optional information mandatory.

40. Hiring application limits

To prevent abuse:

Active applications

Maximum:

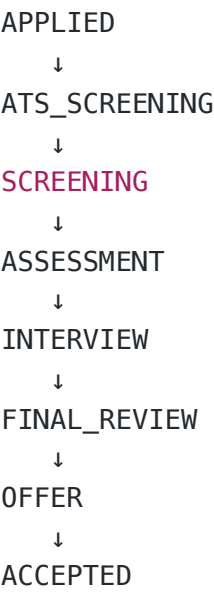
5 active applications simultaneously

User can apply to another company after:

- Rejected
- Withdrawn
- Accepted

41. Hiring stages

Every application follows:



Possible terminal states:

- REJECTED
- WITHDRAWN
- EXPIRED
- ACCEPTED

42. Demo mode

Admin's demo hiring system should use the **same hiring engine**.

That is important.

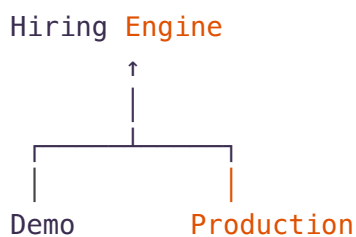
Don't build:

`DemoHiringSystem`

and separately:

`RealHiringSystem`

Instead:



Admin configures:

Domain
Number of questions
Difficulty
Interview type

But the underlying hiring process remains the same.

43. Admin permissions

Admin can:

View users
Edit users
Suspend users
Restore users

- Manage domains
- Manage companies
- Manage jobs
- Configure EXP
- Configure CorpCoin
- Configure warnings
- Configure founder requirements
- Configure demo
- View analytics
- View AI health
- View queues
- View system logs

But dangerous operations should require confirmation.

For example:

- Delete company
- Delete user
- Reset economy
- Force terminate employee

44. AI Manager permissions

AI Manager can:

- Add provider
- Remove provider
- Enable provider
- Disable provider
- Configure provider priority
- Configure model
- Configure limits
- Monitor health
- View AI usage
- View AI failures
- Manage fallback
- Test provider

But AI Manager should **not automatically get Admin permissions**.

45. Audit log

This is something you need to add before coding.

For important actions:

Who
What
When
Target
Old value
New value
Reason

Example:

Admin
changed
founderUnlockExp
9000 → 12000
2026-10-07

Another:

AI Manager
disabled
Gemini
reason: repeated provider failures

This becomes extremely useful when debugging.

46. Database collections

A starting MongoDB structure:

users
profiles
resumes
resumeAnalyses

domains
skills

companies
companyEmployees
companyBots

companyJobs

applications

interviews

assessments

questions

answers

evaluations

feedback

employeeTasks

taskSubmissions

performanceRecords

warnings

promotions

demotions

founders

companyDecisions

companyFinancials

expTransactions

corpCoinTransactions

aiProviders

aiModels

aiRequests

aiResponses

aiJobs

aiHealthLogs

leaderboards

notifications

auditLogs

platformConfigs

You don't necessarily need all of these as separate collections on day one, but this is the conceptual model.

47. Transactions are important

For economic operations:

CorpCoin **purchase**

CorpCoin **deduction**

EXP **award**

EXP **adjustment**
Company **profit**
Company **loss**

Don't simply modify:

```
user.corpCoin -= 250
```

without recording why.

Use a transaction ledger:

corpCoinTransactions

Example:

```
{  
  userId,  
  type: "BOT_PURCHASE",  
  amount: -250,  
  balanceAfter: 750,  
  referenceId: botId,  
  createdAt  
}
```

Same principle for EXP.

48. EXP ledger

Example:

```
{  
  userId,  
  amount: 50,  
  type: "TASK_COMPLETION",  
  sourceId: taskSubmissionId,  
  createdAt  
}
```

Then:

```
total EXP =  
sum(valid EXP transactions)
```

You can also cache the current balance for performance.

49. Security limits

Set these from the beginning.

Login

```
5 failed attempts
```

within a defined window → temporary lock.

Password

Use a secure password hashing algorithm such as:

```
Argon2id
```

API

Rate-limit public endpoints.

File upload

```
PDF/DOCX only  
10 MB maximum
```

Never trust file extension alone.

50. One more critical rule: server is authoritative

This should be written directly into your coding specification:

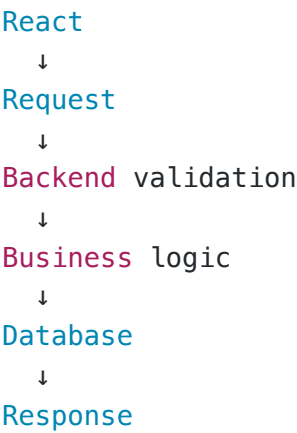
The React frontend is never authoritative.

Frontend can display:

```
EXP
CorpCoin
level
warnings
company balance
```

but cannot decide them.

All important values must be calculated and validated by Node/Express.



51. The final fixed limits for CorpVerse v1

Here is the compact rulebook your coding AI can follow.

System	Rule
Starting role	Job Seeker
Domains	Software, Cloud, AI
Max active applications	5
Resume formats	PDF, DOCX
Resume max size	10 MB
Daily primary tasks	1
Daily bonus tasks	1
Task EXP	10–100

System	Rule
Founder unlock	12,000 EXP
Founder starting CorpCoin	1,000
Company creation	100 CorpCoin
Basic Hiring Bot	250
Basic Task Bot	250
Basic Evaluation Bot	250
Founder companies	1 active
Initial company employees	20
Warning threshold	4 active warnings
Warning expiration	30 days
AI provider attempts	3/provider
Initial AI providers	Gemini + OpenAI + Grok
AI failure behavior	Retry → fallback → queue
Company bankruptcy	-1,000 financial health
Founder daily scenarios	1
Employee primary tasks	1/day
Bonus tasks	1/day
Initial platform companies	3
Maximum level	10

52. What should remain configurable

These should **not** be hardcoded inside business logic:

founderUnlockExp
 starterCorpCoin
 companyCreationCost

botPrices

warningThreshold
warningExpirationDays

taskExpMinimum
taskExpMaximum

applicationLimit

maxEmployees

bankruptcyThreshold

aiRetryCount
aiTimeout
aiProviderPriority

resumeMaxSize

dailyTaskLimit

Put them in:

PlatformConfig

For example:

```
{  
  founder: {  
    unlockExp: 12000,  
    starterCorpCoin: 1000  
  },  
  
  employee: {  
    dailyTaskLimit: 1,  
    bonusTaskLimit: 1,  
    warningThreshold: 4,  
    warningExpirationDays: 30  
  },  
  
  company: {  
    creationCost: 100,  
    maxEmployees: 20,  
    bankruptcyThreshold: -1000  
  },  
  
  applications: {  
    maxActive: 5  
  },  
  
  ai: {
```

```
    retryPerProvider: 3,  
    providers: [  
        "openai",  
        "gemini",  
        "grok"  
    ]  
}  
}
```

53. One important correction to your original concept

You said:

"all main pipeline of any companies are use ai which api set by ai manager"

I would make it:

```
Company  
  ↓  
AI Bot  
  ↓  
AI Gateway  
  ↓  
Provider Router  
  ↓  
Provider Adapter  
  ↓  
Gemini/OpenAI/Grok
```

A company should never know which API is powering its bot.

For example:

```
Founder creates Hiring Bot  
  
Hiring Bot  
  ↓  
AI Gateway  
  ↓  
Currently OpenAI
```

Tomorrow AI Manager changes:

OpenAI → Gemini

The company doesn't change.

That is exactly why your AI-provider abstraction is necessary.

54. And don't make "AI Manager" equal to API keys

The AI Manager is a **CorpVerse platform role**.

The actual API keys should be stored securely in server-side secrets/environment configuration.

Never:

```
React
  ↓
OPENAI_API_KEY
```

Never expose:

```
GEMINI_API_KEY
GROK_API_KEY
OPENAI_API_KEY
```

to the browser.

Correct:

```
React
  ↓
Node/Express
  ↓
AI Gateway
  ↓
Provider
```

55. The rule I would give directly to your

coding AI

This should be included in your project's `ARCHITECTURE.md` or master coding instruction:

Never invent business rules. All gameplay limits, progression requirements, economic values, role permissions, AI retry rules, provider behavior, and state transitions must come from the CorpVerse specification or PlatformConfig. If a required rule is not defined, mark it as TODO and do not silently choose a value. AI providers must be implemented through separate provider adapters because Gemini, OpenAI, and Grok have different APIs, request formats, response formats, models, limits, and error behavior. The AI layer may evaluate or generate recommendations, but the backend remains authoritative for EXP, CorpCoin, roles, promotions, demotions, termination, company finances, and other persistent game state.

That paragraph alone will prevent a lot of **AI-coding hallucination**.

The next artifact I recommend creating

At this point, **don't start generating React/Node code yet.**

The next thing should be a proper:

CORPVERSE_SPECIFICATION.md

with these sections:

1. Product Definition
2. User Roles
3. Career State Machine
4. Authentication Flow
5. Profile & Resume System
6. Job & Company System
7. Hiring State Machine
8. Interview System
9. Employee Task System
10. EXP System
11. Warning / Promotion / Demotion System
12. Founder System
13. CorpCoin Economy
14. Company Simulation Engine
15. Ranking System
16. AI Gateway
17. Gemini Adapter
18. OpenAI Adapter
19. Grok Adapter
20. AI Manager

21. AI Queue & Retry System
22. Admin System
23. Demo System
24. Notification System
25. Audit Logging
26. MongoDB Data Model
27. API Specification
28. Frontend Pages
29. Permission Matrix
30. PlatformConfig
31. Error Handling
32. Security
33. MVP Scope
34. Future Scope
35. Explicit "DO NOT INVENT" Rules